Parte 1

Contexto

La empresa requiere centralizar la información de propiedades inmobiliarias en la ciudad de Medellín. Actualmente, los datos se reciben en un archivo CSV crudo (**propiedades_medellin_raw.csv**) con inconsistencias de formato, duplicados y errores de captura.

El Reto

Tu objetivo es diseñar e implementar un pipeline de datos robusto que transforme este archivo plano en una base de datos relacional normalizada.

1. Modelado de Datos

Diseña un Diagrama Entidad-Relación (DER) para una base de datos relacional.

- El modelo debe estar normalizado (mínimo hasta la 3ra Forma Normal).
- Debe separar atributos en tablas maestras para garantizar la integridad referencial**.**

2. Desarrollo del ETL (Python)

Crea un script en Python que realice las siguientes fases:

- **Extracción**: Lectura del archivo CSV proporcionado.
- **Transformación (Limpieza)**
- **Carga Simulada: * El script debe generar los PRINT de las sentencias SQL (INSERT INTO) necesarias para alimentar el modelo diseñado.**
    - **Control de Duplicidad e Idempotencia:** El código debe estar diseñado para ejecutarse diariamente a las 8:00 AM. Debes implementar una lógica que asegure que si un registro ya existe en la base de datos no se inserte de nuevo.

3. Informe de Calidad de Datos

Adjunta un breve documento (Markdown o PDF) que resuma:

- Principales problemas encontrados en el archivo fuente.
- Lógica aplicada para la limpieza de cada columna crítica.
- Breve explicación de la estrategia utilizada para evitar la carga de datos duplicados en ejecuciones recurrentes.

4. ¿Cómo manejarías si el archivo CSV mañana pesa 10GB?

Parte 2

Contexto

Nuestros asesores inmobiliarios han detectado que el clima es un factor decisivo para los compradores en el Valle de Aburrá. Para mejorar la experiencia de venta, necesitamos integrar información meteorológica en tiempo real a nuestra plataforma.

El Reto

Debes desarrollar un módulo en Python que se conecte a la API de Open – Meteo (https://open-meteo.com/en/docs) para gestionar la información climática de los municipios que conforman el área metropolitana.

1. **Fuentes de Datos:**
   - Crea un archivo maestro (CSV o Diccionario en el código) que contenga los municipios y sus coordenadas base: Envigado, Itagüí, La Estrella, Bello, Copacabana, Sabaneta, Caldas, Girardota, Barbosa y Medellín
2. **Consumo de API:**
   - Para cada municipio, el script debe realizar una petición a la API de Open-Meteo para obtener:
     - Temperatura actual (°C).
     - Código del estado del tiempo (Weather Code).
3. **Lógica de Actualización (Upsert):**
   - El sistema debe ser capaz de actualizar la información cada vez que se ejecute.
   - Si el municipio ya existe en la base de datos, solo se actualice los campos de clima y la "Fecha de última actualización", evitando duplicar el nombre del municipio.
4. **Bonus:** La API de Open-Meteo entrega un campo numérico llamado weathercode (ej. 0, 3, 95). Para nuestros asesores, un número no es útil; ellos necesitan ver una descripción clara del clima en su tablero.